

MQ Player

Full Technical Audit Report

Repository: github.com/killkinhi-a11y/mq-player
Live Site: <https://mq1.vercel.app>
Stack: Next.js 15 (App Router) + TypeScript + Tailwind CSS + Prisma + Web Audio API + HLS.js + Electron

Overall Score: 3.5 / 10

Category	Critical	High	Medium	Low	Total
Security	4	6	8	3	21
Architecture	0	3	4	3	10
Performance	0	1	4	2	7
UI / UX	0	2	6	5	13
Playback	0	2	4	3	9
API Layer	2	5	7	5	19
Total	6	19	33	21	79

1. Executive Summary

This report presents a comprehensive technical audit of the MQ Player project, a music streaming application built on Next.js 15 with App Router, TypeScript, Tailwind CSS, Prisma ORM with PostgreSQL, Web Audio API for playback, HLS.js for streaming, and Electron for desktop packaging. The audit covers the full-stack codebase including 50+ API routes, 30+ React components, 7 interface style themes, a sophisticated audio engine with crossfade and DRM support, and the live deployment at <https://mq1.vercel.app>.

The audit uncovered **79 issues** across 6 categories, with **6 critical** findings that pose immediate security risks. The most severe issues include an open SSRF proxy via the Telegram audio-proxy route, unauthenticated SoundCloud streaming proxies that expose the server to bandwidth abuse and copyright liability, a Telegram webhook endpoint that does not verify request signatures, and a fake encryption system that misleads users into believing their messages are private. Additionally, the live site at mq1.vercel.app auto-redirects visitors to the Telegram bot page (t.me/mqplay_bot) due to AuthView automatically calling `window.open()` after 800ms, making the actual application completely inaccessible without Telegram authentication.

The codebase demonstrates significant ambition with features like AI-powered recommendations, taste profile learning with exponential time decay, dual-audio crossfade, DRM/Widevine EME support, collaborative listening sessions, spatial audio, and 25+ color themes. However, the implementation quality does not match the feature scope. The Zustand store has grown into a monolithic god-object with 80+ actions and 440 lines of interface definitions. The PlayerBar component exceeds 1000 lines. The theme switching system relies on hardcoded class lists that break when new themes are added. Rate limiting uses in-memory storage that is completely ineffective on Vercel serverless. These architectural issues compound the security vulnerabilities and make the codebase difficult to maintain, test, and extend.

2. Critical Security Issues

The following findings represent immediate security risks that should be patched before any further production deployment. Each allows an attacker to compromise the system, exfiltrate data, or abuse server resources with minimal effort.

2.1 Telegram Audio Proxy - Unrestricted SSRF

CRITICAL: The `/api/telegram/audio-proxy` route accepts a url query parameter and fetches it with ZERO domain validation. An attacker can supply any URL including `http://169.254.169.254/latest/meta-data/` (AWS metadata endpoint), `http://localhost:3000/api/admin/stats` (internal API bypass), or any external URL, turning the server into an open proxy. This is classified as a critical SSRF vulnerability.

File: `src/app/api/telegram/audio-proxy/route.ts`

Fix: Validate that the URL hostname is exactly `api.telegram.org`. Reject all other domains.

2.2 Telegram Webhook - No Signature Verification

CRITICAL: The `/api/telegram/webhook` endpoint processes incoming payloads without verifying the Telegram Bot API secret token. Although a `verifyTelegramWebhook()` function exists in `telegram.ts`, it is never called. Any attacker can POST crafted payloads to trigger search queries, playlist creation/deletion, or any action the bot can perform.

File: `src/app/api/telegram/webhook/route.ts`, `src/lib/telegram.ts`

2.3 SoundCloud Proxy Routes - No Authentication

CRITICAL: Five SoundCloud proxy routes (`/stream`, `/proxy`, `/image-proxy`, `/resolve-proxy`, `/license-proxy`) require no authentication. Anonymous users can stream unlimited music through the server, consume bandwidth, exhaust SoundCloud API quotas, and relay DRM license requests. The server acts as an open CDN proxy with no user tracking, creating both bandwidth abuse and copyright liability risks.

Files: `src/app/api/music/soundcloud/*/route.ts`

2.4 Fake Encryption - Messages Are NOT Encrypted

CRITICAL: The `crypto` module uses base64 encoding with a fixed IV prefix, providing zero confidentiality. `simulateDecryptSync()` is trivially reversible by anyone. The `Message` model has `encrypted: true` by default, and the UI likely presents this as real encryption. This is deceptive - users may believe their messages are private when they are not. The `getEncryptionStatus()` function returns "XOR Obfuscation (demo)" but this label is insufficient to convey the actual lack of security.

File: `src/lib/crypto.ts` - `btoa(encodeURIComponent(text))` is NOT encryption

2.5 Admin Email Enumeration

CRITICAL: The `/api/admin/auth` endpoint is publicly accessible and returns whether an email is an admin account. Combined with the hardcoded fallback admin email `killkin.hi@gmail.com` in the source code, an attacker immediately knows a valid admin email. This enables targeted phishing attacks.

File: `src/app/api/admin/auth/route.ts`

2.6 SSRF in SoundCloud Proxies - Insufficient URL Validation

CRITICAL: The proxy routes use `h.endsWith("sndcdn.com")` for domain validation, which can be bypassed with subdomain registration attacks (e.g., `evilsndcdn.com`). The `resolve-proxy` only checks `endsWith("soundcloud.com")`, allowing attacker.`soundcloud.com` if DNS is pointed at internal services. Use explicit hostname allowlists instead.

Files: `src/app/api/music/soundcloud/proxy/route.ts`, `resolve-proxy/route.ts`

3. Architecture Problems

3.1 Monolithic Zustand Store

The `useAppStore.ts` file contains approximately 440 lines of interface definitions and 80+ actions covering player state, messenger state, radio mode with complex recommendation logic, smart shuffle with energy estimation, taste profile management, cat mascot state, EQ state, sleep timer, feedback batching, and collaborative listening. The `nextTrack` function alone is approximately 250 lines with nested conditionals for radio mode, smart shuffle, and taste profile. This violates the single responsibility principle and makes the store extremely difficult to test, debug, and maintain. Split into domain-specific slices: `playerStore`, `messengerStore`, `radioStore`, `uiStore`, etc.

3.2 In-Memory Rate Limiting on Serverless

The rate limiter uses `new Map()` stored in process memory. On Vercel serverless, each function invocation may run in a separate container, so the `Map` resets on every cold start and multiple instances share no state. An attacker simply needs to send requests fast enough to spin up new instances, making rate limiting completely ineffective. The code acknowledges this: "In-memory store (per-process; resets on cold start)" but this is not acceptable for security. Replace with Redis/Upstash for serverless-compatible global rate limiting.

3.3 `requireAuth()` Does Not Enforce Auth

The `requireAuth()` function in `get-session.ts` is identical to `getSession()` - it returns null instead of throwing or returning an error response. The JSDoc says "returns session or throws 401 response" but it returns null, forcing every caller to manually check and return 401. This leads to inconsistent auth checks across the codebase.

3.4 Process Error Handlers Swallow Fatal Errors

`db.ts` installs `process.on("uncaughtException")` and `process.on("unhandledRejection")` handlers that only log errors without terminating the process. In production, this masks critical bugs that should cause the process to restart. Uncaught exceptions typically indicate a corrupted state that should not continue running.

3.5 CSP Allows `unsafe-inline` and `unsafe-eval`

The Content Security Policy in `next.config.ts` includes `script-src` with `unsafe-inline` and `unsafe-eval`, which completely defeats CSP XSS protection. The `unsafe-eval` directive is especially dangerous as it allows arbitrary code execution via `eval()`. These are likely needed for some dependencies but should be replaced with nonces or hashes.

3.6 Dual Deployment Confusion

The project has deployment configurations for Vercel, Cloudflare (`wrangler`, `_headers`), and Electron simultaneously. The `public/_headers` file is Cloudflare-specific but only has one rule. The Vercel security

headers in `next.config.ts` are more comprehensive. This creates maintenance burden and potential configuration conflicts between platforms.

4. Interface and UI Audit

4.1 Live Site Inaccessible - Auto-Redirect to Telegram

CRITICAL: The live site at <https://mq1.vercel.app/play> automatically redirects visitors to t.me/mq1play_bot because `AuthView.tsx` calls `window.open("https://t.me/" + botName)` after 800ms on mount (line 64-66). This makes the actual MQ Player application completely inaccessible to anyone visiting the URL directly. Users cannot explore the app, see the interface, or use the demo mode before being redirected away. The browser test confirmed this - the page loads the Next.js app, then immediately navigates to the Telegram bot page.

Fix: Remove the auto-redirect. Let users click the "Open Bot" button manually. Show the demo mode option prominently.

4.2 Component Size and Complexity

The `PlayerBar` component exceeds 1000 lines including EME/DRM handling, error recovery, HLS configuration, and crossfade management. The `MainView` component is also extremely large with AI recommendations, curated playlists, taste profile building, and multiple data-fetching effects. These components should be decomposed into smaller, focused sub-components. The `PlayerBar` alone contains: `buildEmeHlsConfig()`, `setupManualEME()`, `buildWidevinePssh()`, `createManifestInterceptor()`, `resolveSoundCloudStream()`, `ShareButton`, `MagneticPlayButton`, and the main `PlayerBar` - each of these should be its own module.

4.3 Accessibility Issues

While some components have good ARIA labels (`NavBar` has `aria-label` and `aria-current`), many interactive elements lack proper accessibility support. Track cards use `motion.button` without `aria-labels`, the progress bar in `PlayerBar` uses a `div` with click handlers but no proper slider role or keyboard interaction, the volume control similarly lacks keyboard support, and the `FullTrackView` overlay does not trap focus. Touch targets on mobile are frequently smaller than the recommended 44x44px minimum, particularly the playback control buttons in compact mode and the code input fields in `AuthView`. The 6-digit code inputs have proper `inputMode="numeric"` but lack `autocomplete="one-time-code"` consistently and have no `aria-describedby` for error messages.

4.4 Theme System Fragility

The `themes.ts` `applyThemeToDOM()` function contains a hardcoded array of 24 theme class names that must be manually updated when new themes are added. If a developer adds a theme without updating this list, the old theme class will persist on the HTML element, causing visual conflicts. The theme system also has two competing mechanisms: CSS custom properties (`--mq-*`) set by JavaScript, and Tailwind CSS classes (`hsl(var(--))`) configured in `tailwind.config.ts`. With Tailwind v4 migration, the `tailwind.config.ts` may be partially ignored, creating confusion about which definitions are active. The 7 interface styles (`streaming.css`, `japan.css`, `neon.css`, etc.) use `data-style` attributes on the HTML element, which is a reasonable approach, but the interaction between `currentStyle` and `currentTheme` is unclear - when `currentStyle` is set, theme variables are skipped entirely, which can leave stale CSS variable values.

4.5 Visual Effects Performance

The application uses multiple visual effects simultaneously: HeroParticles (canvas), CursorSpotlight (mouse-following), ScrollReveal (intersection observer), ScrollProgressBar, SeasonalEffects, DNAHelixVisual, and TrackCanvas (audio visualizer). Each of these creates its own animation loop or event listener. The TrackCanvas visualizer runs at 60fps with requestAnimationFrame even when the mini-player is hidden. The audioEngine has adaptive performance quality but the canvas rendering in PlayerBar does not check the performance level before rendering. On low-end devices, these concurrent animations can cause significant jank and battery drain.

4.6 Loading and Hydration Issues

The play/page.tsx uses a useIsClient() pattern that renders a static splash on the server, then replaces it with the full AppShell on the client. This causes a visible flash of content (FOUC) as the entire page structure changes on hydration. The splash screen created by the inline script in layout.tsx partially mitigates this, but the splash auto-removes after 2.5 seconds regardless of whether the app has finished loading. Additionally, the db-sync endpoint is called on every page load, creating an unnecessary database round-trip before the app can render its main content.

5. Playback and Audio Engine

5.1 Audio Engine Design

The `audioEngine.ts` implements a dual-audio-element crossfade system with Web Audio API integration. Two `HTMLAudioElement` instances share one `AudioContext` via `GainNodes`, with an `AnalyserNode` for visualization and a 5-band EQ filter chain. The design is fundamentally sound and demonstrates sophisticated audio engineering including adaptive performance levels (high/medium/low based on frame time monitoring), simulated frequency data fallback when CORS blocks real data, and Firefox-specific EME compatibility through element replacement. However, several issues affect reliability.

5.2 Race Conditions in Crossfade

The `crossfadeTo()` function captures audio element references (`_audioA`, `_audioB`) in a `setTimeout` callback. If `replaceAudioElement()` is called between the crossfade initiation and the timeout firing (during Firefox EME replacement), the callback operates on stale elements. Additionally, multiple rapid calls to `crossfadeTo()` can leave stale `linearRampToValueAtTime` events that override new values, causing volume jumps. The code attempts to mitigate this with `cancelScheduledValues()` but this is not always sufficient when the active/inactive swap has already occurred.

5.3 DRM/EME Complexity

The `PlayerBar` contains approximately 400 lines of EME/DRM handling code including manual Widevine setup, PSSH box construction from manifest KEYID, Firefox element replacement for EME compatibility, and a 5-second PSSH fallback timer. This is extremely complex and fragile. The code has two competing EME strategies: HLS.js built-in EME controller (`emeEnabled: true`) and manual EME setup (`setupManualEME` function). The comments indicate the manual approach was previously tried but failed because "HLS.js uses MSE internally - the encrypted event never fires on the audio element." This suggests the current HLS.js EME approach may also have edge cases that fail silently. The extensive diagnostic logging throughout (`console.log` for license URLs, auth token lengths, CDM challenge bytes) should be removed from production builds as it leaks sensitive information.

5.4 SoundCloud Streaming Reliability

The `resolveSoundCloudStream()` function tries multiple SoundCloud client IDs sequentially when resolving tracks. With 3 client IDs and a 10-second timeout per attempt, the worst case is 30 seconds before returning, approaching Vercel edge function timeout limits. The function also exposes diagnostic information (`_diag` array) in production responses, including internal details about API call status codes and resolved URLs. The fallback stream mechanism is a good idea but the retry logic in `PlayerBar` can create cascading timeouts that freeze the UI for extended periods.

5.5 localStorage Overflow

The Zustand store persists to `localStorage` with `createJSONStorage`. The store includes history (can grow indefinitely), `trackFeedback` (grows per-track), `feedbackBatch` (accumulates data),

likedTracksData and dislikedTracksData (full track objects with covers, genres, etc.), publicPlaylists, and recommendedPlaylists. There is no size limit on localStorage and no eviction policy. The store will eventually hit the 5-10MB localStorage limit, causing silent failures or performance degradation. A heavy user with 500+ tracks in history and full track objects can easily exceed 5MB.

6. API Layer Security

6.1 Authentication Gaps

Multiple API routes lack authentication or have inconsistent auth checks. The SoundCloud proxy routes (5 endpoints), the Telegram audio-proxy, the Telegram webhook, the SoundCloud diagnose endpoint, and the stories feed (all=true) all operate without authentication. Even authenticated routes have issues: the requireAuth() helper does not enforce auth, the admin auth endpoint leaks admin email status, and verification codes can be brute-forced with distributed IPs (6-digit codes with 10-attempt/minute rate limit are vulnerable to 10+ IP botnet attacks).

6.2 Input Validation Gaps

Message content has no length limit, allowing multi-megabyte text payloads. Client-provided IDs are used as database primary keys, which could cause collisions. Password validation has no maximum length, allowing bcrypt DoS with extremely long passwords. Email regex is too permissive. File upload MIME type checking uses client-provided file.type without magic byte validation. Voice URLs in messages are not validated. These gaps allow both accidental and intentional data corruption and denial-of-service attacks.

6.3 Data Leakage

The login endpoint returns different error messages for non-existent emails vs wrong passwords, enabling email enumeration. The send-code endpoint similarly reveals whether an email is registered. In development mode, verification codes are leaked in API responses via the devCode field, and codes are logged to console when email is not configured. The admin stats endpoint returns user emails in recentRegistrations. Error handlers in music routes return 200 with empty arrays, masking real errors and making debugging impossible while simultaneously leaking internal error details in upload and other routes.

6.4 Route-by-Route Findings

Severity	Finding	File(s)	Description
CRITICAL	Telegram SSRF	telegram/audio-proxy	Zero domain validation on url parameter
CRITICAL	Webhook no sig verify	telegram/webhook	verifyTelegramWebhook() never called
CRITICAL	Admin email enum	admin/auth	Public endpoint reveals admin status
HIGH	SC stream no auth	soundcloud/stream	Open streaming proxy, bandwidth abuse
HIGH	SC diagnose no auth	soundcloud/diagnose	Leaks client IDs, CDN URLs, metadata
HIGH	License proxy no auth	soundcloud/license-proxy	Open DRM license relay
HIGH	Code brute force	auth/verify-code	6-digit codes vulnerable to botnet
HIGH	Email enumeration	auth/login, send-code	Different messages reveal registered emails
HIGH	No message validation	messages/route	No content length limit, client-provided ID
MEDIUM	No password max length	auth/register	bcrypt DoS with long passwords

Severity	Finding	File(s)	Description
MEDIUM	CORS reflects origin	license-proxy	Access-Control-Allow-Origin: origin ""
MEDIUM	Errors return 200	music/* routes	All errors return empty arrays with 200 status
MEDIUM	Error details leak	music/upload	Internal error messages exposed to clients
LOW	Case-sensitive search	playlists/route	contains without mode: insensitive
LOW	Temp password in response	admin/users	Password reset returns temp password

7. Configuration and Build Issues

7.1 Next.js Configuration

- `generateBuildId: Date.now()` generates new build ID on every build, invalidating all CDN/browser caches
- CSP with `unsafe-eval` defeats XSS protection entirely
- `reactStrictMode` is `false` - disables important React development warnings
- `ignoreBuildErrors` may hide TypeScript compilation errors in production builds
- Aggressive cache-busting in `layout.tsx` clears all `localStorage` on version change

7.2 TypeScript Configuration

- `noImplicitAny: false` weakens type safety - parameters silently get any type
- Several any types in Zustand store (`listenSession: null as any`, `setListenSession: (session: any) => void`)
- Missing fields in `AppState` interface vs `initialState` definition

7.3 Tailwind CSS Version Conflict

The project uses Tailwind v4 (based on `package.json` and CSS-first `@theme` inline in `globals.css`) but the `tailwind.config.ts` file uses v3-style `hsl(var(--))` syntax. With Tailwind v4, this config file may be partially or completely ignored, creating confusion about which definitions are active. The `globals.css` file is 987 lines with two complete theme systems, duplicate keyframe animations, and `@import` statements at the bottom of the file (should be at top per CSS spec).

7.4 Prisma Schema

- `VerificationCode` stores code as plain text - should be hashed
- `User.role` is `String`, not `enum` - no database-level constraint
- `Message.content` has no `@db.Text` or length constraint
- `TrackComment` stores denormalized username/avatar - no cascade update
- Missing `onDelete`: Cascade on `Message`, `Friend`, `Notification` relations

7.5 Environment and Secrets

- `NEXT_PUBLIC_ADMIN_EMAILS` exposes admin email in client bundle
- SoundCloud client IDs hardcoded in 4+ files instead of environment variables
- Admin email `killkin.hi@gmail.com` hardcoded as fallback in `admin/auth` route
- Electron `main.js` hardcodes production URL `https://mq1.vercel.app`

8. Improvement Recommendations

8.1 Immediate Fixes (P0 - Ship Blockers)

Security

- Add hostname allowlist validation to telegram/audio-proxy - only allow api.telegram.org
- Call verifyTelegramWebhook() at the top of the POST handler in telegram/webhook
- Remove or protect /api/admin/auth endpoint - never expose admin email lists publicly
- Add authentication to all SoundCloud proxy routes (at minimum require a valid session cookie)
- Replace endsWith() domain checks with explicit hostname allowlists in all proxy routes
- Remove the fake encryption or clearly label it as "encoding" not "encryption" in the UI
- Move SoundCloud client IDs to environment variables and centralize into a single module

Site Access

- Remove auto-redirect window.open("https://t.me/...") from AuthView - let users click the button manually
- Show the demo mode button prominently so users can explore the app without Telegram

8.2 Short-Term Fixes (P1 - Next Sprint)

Security

- Add per-code attempt tracking with lockout after 5 failed verification attempts
- Use generic "Invalid credentials" / "If an account exists, a code has been sent" messages
- Add message content validation (max length, type checking, receiverId format)
- Add password max length check (1024 characters)
- Remove devCode feature or use a separate EXPOSE_CODES env var, not NODE_ENV

Architecture

- Split Zustand store into domain slices: playerStore, messengerStore, radioStore, uiStore
- Fix requireAuth() to throw 401 or return error response, not null
- Remove process.on("uncaughtException") error swallowing
- Add localStorage size limits and eviction policy to Zustand persist

8.3 Medium-Term Improvements (P2 - Next Sprint)

Performance

- Replace in-memory rate limiting with Upstash Redis for serverless compatibility
- Remove generateBuildId: Date.now() from next.config.ts
- Add request timeouts to SoundCloud stream route (prevent 30s worst case)
- Implement image proxy streaming instead of loading entire file into memory

- Add adaptive visualization quality check in PlayerBar canvas rendering
- Implement lazy loading for visual effects (only activate when visible)

UI/UX

- Decompose PlayerBar and MainView into smaller sub-components
- Add proper focus management for modals and full-screen views
- Ensure all touch targets are minimum 44x44px
- Add proper ARIA labels to all interactive elements
- Implement skeleton loading states instead of spinner
- Fix hydration flash with proper SSR strategy

Code Quality

- Set noImplicitAny: true in tsconfig.json
- Remove unsafe-eval from CSP and use nonce-based script loading
- Remove production console.log statements from all routes
- Move CSS @import statements to top of globals.css
- Add type-safe listenSession interface instead of any
- Fix hardcoded theme class list in applyThemeToDOM - derive dynamically

8.4 Long-Term Improvements (P3 - Backlog)

- Implement real E2E encryption for messages using Web Crypto API
- Replace base64 avatar storage with S3/R2 object storage
- Add structured logging with request IDs for audit trails
- Implement API key management system for SoundCloud credentials
- Add schema-level constraints (User.role enum, content length limits)
- Unify deployment target (pick Vercel OR Cloudflare, not both)
- Consider replacing SQLite/PostgreSQL dual approach with a single managed database
- Add comprehensive E2E testing with Playwright for critical user flows
- Implement content-type validation on proxy responses
- Add CSRF protection for state-changing operations

9. Score Breakdown

Category	Critical	High	Medium	Low	Total
Security	4 critical SSRF/auth vuln	2/10 vulnerabilities, fake encryption, email enumeration			
Architecture	God store, broken rate limiting	4/10 error swallowing, dual deployment confusion			
Performance	Adaptive audio engine, but 5/10	5/10 Storage overflow, no streaming proxy, heavy animations			
UI / UX	Rich features and theme files	5/10, but inaccessible site, poor a11y, component bloat			
Playback	Sophisticated engine with crossfade	6/10 DRM/EQ, but race conditions and reliability issues			
API Layer	Wide feature set but critical auth gaps	8/10 SSRF, no input validation, data leakage			
Code Quality	no implicitAny off, any types	4/10 dead code, 987-line CSS, hardcoded configs			
Overall	Ambitious feature set	8/10	defined by critical security flaws and architectural debt		

The MQ Player project demonstrates remarkable ambition with features like AI-powered recommendations with exponential time decay, dual-audio crossfade, Widevine DRM support, spatial audio, 25+ color themes, 7 interface styles, collaborative listening, and a Telegram-based authentication flow. The audio engine is particularly sophisticated with adaptive performance levels and simulated frequency fallback. However, the implementation quality does not match the feature scope. Critical security vulnerabilities, a monolithic state management architecture, and a non-functional live deployment overshadow the technical achievements. The project needs a focused security hardening sprint, architectural refactoring of the Zustand store and PlayerBar, and a fix for the site accessibility before any further feature development.